

CS1023 Final Project Report

Java-Based Arbitrary Precision Arithmetic Calculator

Course Name: Software Development Fundamentals

Submitted by:

Dara Pooja

CS24BTECH11021

Indian Institute of Technology Hyderabad

1 Abstract

This project implements an arbitrary precision arithmetic library in Java to perform basic arithmetic operations (addition, subtraction, multiplication, and division) on integers and floating-point numbers of arbitrary size and precision. The library consists of two main classes: `AInteger` for large integers and `AFloat` for high-precision floating-point numbers. Using operator overloading, these classes handle operations beyond the range of Java's native data types. The project demonstrates the application of object-oriented programming (OOP) concepts and provides a robust solution for calculations requiring high precision. The report covers the design, implementation, usage, and testing of the library, as well as the integration of build tools like Ant and Python.

2 Introduction

This project implements an arbitrary-precision arithmetic library, enabling operations on integers and floating-point numbers without the limitations of standard Java data types such as `int`, `long`, `float` and `double`. It allows calculations on numbers of any size or precision with high accuracy.

The main components of the project are:

- **AInteger Class** : Handles arithmetic operations for arbitrary-precision integers.
- **AFloat Class** : Handles arithmetic operations for arbitrary-precision floating-point numbers.
- **MyInfArith Class** : The driver class that manages user input and performs the requested operations.
- **Ant Build Script** : Automates the building, compiling, and running of the project
- **python Script** : Provides a simple command-line interface to compile and execute the project easily.

2.1 Standard Types vs. Custom Arbitrary-Precision Types

Java's built-in numeric types have fixed ranges, which limit their ability to handle very large numbers or precise floating-point calculations. The ranges of these types are:

- **int**: A 32-bit signed integer, with values ranging from -2^{31} to $2^{31} - 1$.
- **long**: A 64-bit signed integer, with values ranging from -2^{63} to $2^{63} - 1$.
- **float**: A 32-bit floating-point number, capable of representing values approximately from $\pm 3.4 \times 10^{38}$, with 7 digits of precision.
- **double**: A 64-bit floating-point number, capable of representing values approximately from $\pm 1.8 \times 10^{308}$, with 15-16 digits of precision.

These built-in types are useful but cannot handle calculations requiring much larger numbers or higher precision. In contrast, the **AInteger** and **AFloat** classes in this project remove these limitations. They allow for the handling of numbers with arbitrary size and precision, making them ideal for high-precision arithmetic and large-scale calculations.

3 Project Features

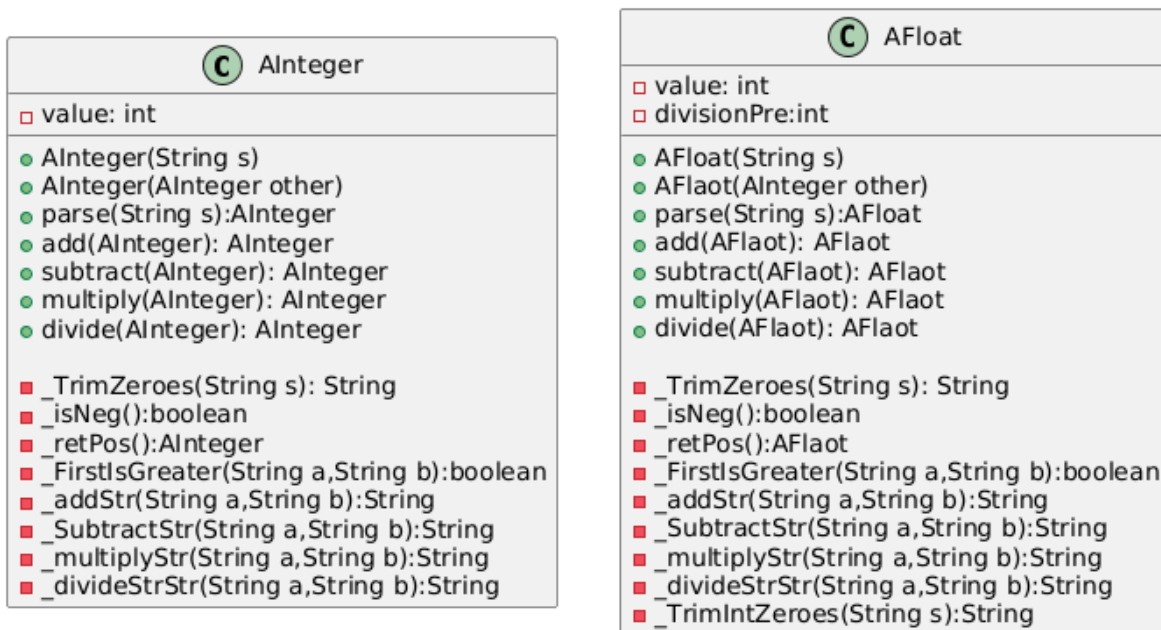
A brief description of the main components:

- **MyInfArith** : The **MyInfArith** class is not located in the same directory as **AInteger** and **AFloat**, but it is used to execute the given test cases. Let's take a look at this in the 'Build and Run' section and 'file description' section.
- **Ant file**: The **Ant** file contains targets for **compile**, **clean**, **jar**, and **run**. Let's also take a look at this in the 'Build and Run' section and 'file description' section .
- **AInteger.java** : The **AInteger** class is designed to represent integer values with additional features for performing mathematical operations such as addition, subtraction, multiplication, and division. This class also includes input validation to ensure only valid integer values are processed and allows normalization of the integer values to a specific range.
 - In this implementation, the numbers are input as strings, which allows for arbitrary length.
 - In this the operations are performed digit to digit.
- **AFloat.java** : The **AFloat** class is designed to represent float values with additional features for performing mathematical operations similar to **AInteger** class.

- In this implementation, the numbers are input as strings, which allows for arbitrary length.
- In this the operations are performed digit to digit.

In both classes, `StringBuilder` is used to construct the result digit by digit. This approach is memory-efficient compared to string concatenation. Initially, using plain string operations led to an `OutOfMemoryError`, which was resolved by switching to `StringBuilder`.

3.1 UML Diagram of AInteger and AFloat Classes



4 File description

4.1 MyInfArith

The `MyInfArith` class serves as the entry point for running the arbitrary-precision arithmetic operators. It accepts command-line arguments to specify the operation (such as addition, subtraction, multiplication, or division) and the operands (either integers or floating-point values). Depending on the type of operation, the corresponding class (`AInteger` or `AFloat`) is used to perform the calculation.

- Depends on the operation required and type of operands the required operation is invoked from `AInteger` or `AFloat`.

4.2 Ant Build file(build.xml)

The Ant file automates compilation, clean, run test case and creating jars using the following targets:

- **Compile** : Compile all java source files.
- **Clean** : To delete the compiled files generated by the 'compile' target (which forms the 'build' directory) and the JAR file created by the 'jar' target (which forms the 'lib' directory), use the 'clean' target.
- **jar** : Packages the compiled classes into a JAR file for easier distribution and execution.
- **run** : Executes the MyInfArith class, passing the appropriate arguments to perform the arithmetic operations.

4.3 runproject.py:

- A Python script to compile and run MyInfArith.java
- Accepts 4 command-line arguments: type, operation, operand1, and operand2.
- Uses subprocess.run() to handle Java compilation and execution.
- Displays result or error based on Java program output.

5 Testing

Correctness of the functions of these classes was mainly tested manually, with comprehensive test cases for both integer and float operations.

5.1 AInteger

1. $12345 + 67890 = 80235$
2. $50000 + (-20000) = 30000$
3. $0012345 + 067890 = 80235$
4. $100000 - 23456 = 76544$

6.2 Executing through .jar file

If we run **ant jar** The jar file is created.

A JAR file (Java ARchive) is a package file format used to bundle multiple Java class files, associated metadata, and resources (such as images, configuration files, etc.) into a single file. It's essentially a zip file with a .jar extension that is used for storing and distributing Java applications or libraries.

And it serves the following purposes:

- JAR files allow developers to bundle all components of a Java application or library into one single file, simplifying distribution and deployment.
- The JAR file is created through an automated build process defined in the build.xml Ant file. This file compiles the Java classes and packages them into the JAR.

To run test-cases we should use:

- ant jar
- java -cp lib/aarithmetic.jar MyInfArith <type> <operation> <operand1> <operand2>

6.3 Executing through run_project.py

Python script that compiles and runs the Java project using command-line arguments

Help automate command-line execution in a repeatable way.

Through this I understood how to use Python for scripting and automation.

- python run_project.py -compile
- python run_project.py -run <type> <operation> <operand1> <operand2>

7 Conclusions

- Arbitrary Precision Arithmetic:
 - Handling numbers beyond standard data types (int, double).
 - Implementing operator overloading for large numbers.
- Object-Oriented Programming:

- Designing classes (AInteger, AFloat) and encapsulating logic.
 - Using constructors, copy constructors, and static methods.
- Build Automation (Ant):
 - Automating compilation, packaging, and testing using Ant.
 - Understanding the importance of build tools in large projects.
- Integration of Tools:
 - Combining Ant, Python, and Java for streamlined project management
- How to use .Jar packages in files
- using python to build execution scripts.